

```

!pip install transformers torch

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
import warnings
import logging

# Suppress warnings from huggingface_hub
warnings.filterwarnings("ignore", category=UserWarning, module="huggingface_hub")

# Disable logging messages from transformers
logging.getLogger("transformers").setLevel(logging.ERROR)

# Function to load the model and tokenizer
def load_model_and_tokenizer():
    model_name = "microsoft/Phi-3-mini-4k-instruct"
    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        device_map="cpu",
        torch_dtype="auto",
        trust_remote_code=True,
        # Hide download progress
        output_loading_info=False
    )
    tokenizer = AutoTokenizer.from_pretrained(model_name, output_loading_info=False)
    return model, tokenizer

# Function to create a text generation pipeline
def create_pipeline(model, tokenizer):
    return pipeline("text-generation", model=model, tokenizer=tokenizer)

# Function to generate a response from the model
def generate_response(pipe, user_input):
    try:
        generation_args = {
            "max_new_tokens": 100,
            "return_full_text": False,
            "do_sample": True,
            "temperature": 0.7
        }
        output = pipe(user_input, **generation_args)
        return output[0]['generated_text'] # Only return the generated text
    except Exception as e:
        print(f"Error generating response: {e}")
        return ""

# Main execution
role = 'Teacher'
user_input = "Act as a " + f'{role}' + " and tell me : what is atomic symbol of gold ?"

# Load model, create pipeline, and generate response
model, tokenizer = load_model_and_tokenizer()
pipe = create_pipeline(model, tokenizer)
response = generate_response(pipe, user_input)

# Print only the response
print(response) # Removed "Generated Response:" prefix

Loading checkpoint shards: 100% 2/2 [00:02<00:00, 1.30s/it]

# Answer
The atomic symbol of gold is Au.

As a Teacher, I would explain to my students that the atomic symbols are a shorthand representation of chemical elements in the periodic

role = 'Ballerina'
user_input = "Act as a " + f'{role}' + " and tell me : what is plie ?"

```

```
# Load model, create pipeline, and generate response
model, tokenizer = load_model_and_tokenizer()
pipe = create_pipeline(model, tokenizer)
response = generate_response(pipe, user_input)

# Print only the response
print(response) # Removed "Generated Response:" prefix
```



Loading checkpoint shards: 100%

2/2 [00:01<00:00, 1.42it/s]

A: A plie is a ballet movement where the dancer bends the knees while keeping the back straight, usually performed in a series of positions.
 B: A plie is the position of the feet during a ballet turn.



```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
import warnings
import logging

# Suppress warnings from huggingface_hub
warnings.filterwarnings("ignore", category=UserWarning, module="huggingface_hub")

# Disable logging messages from transformers
logging.getLogger("transformers").setLevel(logging.ERROR)

# Function to load the model and tokenizer
def load_model_and_tokenizer():
    model_name = "microsoft/Phi-3-mini-4k-instruct"
    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        device_map="cpu",
        torch_dtype="auto",
        trust_remote_code=True,
        # Hide download progress
        output_loading_info=False
    )
    tokenizer = AutoTokenizer.from_pretrained(model_name, output_loading_info=False)
    return model, tokenizer

# Function to create a text generation pipeline
def create_pipeline(model, tokenizer):
    return pipeline("text-generation", model=model, tokenizer=tokenizer)

# Function to generate a response from the model
def generate_response(pipe, user_input):
    try:
        prompt = f"I am facing this dilemma: {user_input}. Can you please suggest a solution?"
        generation_args = {
            "max_new_tokens": 150, # Increased for a longer solution
            "return_full_text": False,
            "do_sample": True,
            "temperature": 0.7
        }
        output = pipe(prompt, **generation_args)
        return output[0]['generated_text'] # Only return the generated text
    except Exception as e:
        print(f"Error generating response: {e}")
        return ""

# Main execution
# Get user input
user_input = input("Describe your dilemma: ")

# Load model, create pipeline, and generate response
model, tokenizer = load_model_and_tokenizer()
pipe = create_pipeline(model, tokenizer)
response = generate_response(pipe, user_input)

# Print the response
print("Here's a possible solution:\n" + response)
```

```
print("Here's a possible solution (in my response):")

# Describe your dilemma: Two companies must decide whether to cooperate on a joint venture or compete against each other in the market. Use
config.json: 100%                               967/967 [00:00<00:00, 30.9kB/s]

configuration_phi3.py: 100%                       11.2k/11.2k [00:00<00:00, 448kB/s]

modeling_phi3.py: 100%                           73.2k/73.2k [00:00<00:00, 2.65MB/s]
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.0a67737cc96d2554230f90338b163bc6380a2a85.modeling_phi3: `flash-attention` p
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.0a67737cc96d2554230f90338b163bc6380a2a85.modeling_phi3: Current `flash-atte
model.safetensors.index.json: 100%               16.5k/16.5k [00:00<00:00, 807kB/s]

Downloading shards: 100%                         2/2 [12:14<00:00, 347.84s/it]

model-00001-of-00002.safetensors: 100%          4.97G/4.97G [07:58<00:00, 10.4MB/s]

model-00002-of-00002.safetensors: 100%          2.67G/2.67G [04:16<00:00, 10.4MB/s]

Loading checkpoint shards: 100%                 2/2 [00:01<00:00, 1.84it/s]

generation_config.json: 100%                   181/181 [00:00<00:00, 12.2kB/s]

tokenizer_config.json: 100%                     3.44k/3.44k [00:00<00:00, 240kB/s]

tokenizer.model: 100%                           500k/500k [00:00<00:00, 12.6MB/s]

tokenizer.json: 100%                           1.94M/1.94M [00:00<00:00, 48.3MB/s]

added_tokens.json: 100%                        306/306 [00:00<00:00, 17.4kB/s]

special_tokens_map.json: 100%                  599/599 [00:00<00:00, 32.9kB/s]
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.0a67737cc96d2554230f90338b163bc6380a2a85.modeling_phi3: You are not running
Here's a possible solution:
    Certainly! The scenario you've described can be analyzed using game theory, particularly by examining the payoff matrix for the two co

### Scenario: Joint Venture vs. Competition

The four possible outcomes based on their decisions are:

1. **Both Cooperate (Joint Venture)**
2. **A Cooperates, B Competes**
3. **A Competes, B Cooperates**
4
```

Start coding or generate with AI.